

Credit Name: File Structures & Exception Handling
Assignment Name: StudentSemesterAverage.java

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem:

How did you approach understanding the challenge?

understood the challenge by splitting it into small parts: getting student information, finding the average, and saving it to a file. Once I knew the main steps, it was easier to build each part of the program one at a time.

```
public void actionPerformed(ActionEvent e) {
    String StuNres = Stures.getText(); // get student name
    String sturgraderesRES = sturgraderes.getText(); // get student grade
    String SNR = semnumres.getText(); // get semester
    String g1 = G1Res.getText(); // get grade 1
    String g2 = G2res.getText(); // get grade 2
    String g3 = G3Res.getText(); // get grade 3
    String g4 = G4Res.getText(); // get grade 4

    double semavg = (Double.parseDouble(g1) + Double.parseDouble(g2) +
        Double.parseDouble(g3) + Double.parseDouble(g4)) / 4; // calculate average
```

This was getting student information

```
double semavg = (Double.parseDouble(g1) + Double.parseDouble(g2) +
    Double.parseDouble(g3) + Double.parseDouble(g4)) / 4; // calculate average

AVGdisp.setText(Double.toString(semavg)); // show average
```

This was getting the average of the student in whichever grade he inputs

```
    " Average: " + semavg; // create file line
try {
    FileWriter fw = new FileWriter("../Chapter11/src/Mastery/students.txt", true); // open file
    PrintWriter pw = new PrintWriter(fw); // connect print writer
    pw.println(Studentinfo); // write line
    pw.close(); // close file
}
catch(IOException ex) {
    ex.printStackTrace(); // print errors
}
});

JButton Viewfile = new JButton("View File contents"); // view button
Viewfile.setBounds(234, 389, 196, 58); // position
frame.getContentPane().add(Viewfile); // add

Viewfile.addActionListener(new ActionListener() { // view click
    public void actionPerformed(ActionEvent e) {
        try {
            FileReader fr = new FileReader("../Chapter11/src/Mastery/students.txt"); // open file
            BufferedReader br = new BufferedReader(fr); // buffered reader

            String line; // current line
            while ((line = br.readLine()) != null) {
```

This was displaying the student's information through a file

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

Yes, the most confusing part at first was understanding how the file saving and reading worked. I was unsure how the program could store the student information in a text file and then show it again in the text area. I resolved this by looking back at the Chapter 11 examples and skill builders, which helped me understand how `FileWriter`, `PrintWriter`, `FileReader`, and `BufferedReader` work together. Once I tested it and saw the file update correctly, it made much more sense.

```
try {
    FileReader fr = new FileReader("../Chapter11/src/Mastery/students.txt"); // open file
    BufferedReader br = new BufferedReader(fr); // buffered reader

    String line; // current line
    String allText = ""; // all file text

    while((line = br.readLine()) != null) { // read until end
        allText += line + "\n"; // add line
    }

    Studentinfolist.setText(allText); // display in text area

    br.close(); // close reader
}
catch(IOException ex) {
    ex.printStackTrace(); // print errors
}
```

At first it didn't wrap around my head on how a GUI can read files because I thought only class could but once I started doing it, once I started doing it and it worked it felt easier to process the information.

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

Yes, I broke the problem into smaller steps before coding. First, I planned the GUI layout with all the text fields and buttons. Then I worked on calculating the semester average, after that saving the data to the file, and finally reading the file back into the text area. Doing it step by step made the program easier to build and test.

```
double semavg = (Double.parseDouble(g1) + Double.parseDouble(g2) +
    Double.parseDouble(g3) + Double.parseDouble(g4)) / 4; // calculate average

AVGdisp.setText(Double.toString(semavg)); // show average
```

Doing this was the frame work of the code because this has to be stored in the files and if it doesn't work it would of had many errors and not completed the scope of the project

How did you decide on the tools, data structures, or algorithms to use?

I decided on the tools by matching each part of the problem to what Java does best. I used Swing components like text fields, labels, buttons, and a text area because the program needed a GUI. For storing the student record, I used simple String variables since the information was just text being written to a file. The main algorithm was straightforward: read the four grades, convert them to numbers, calculate the average by adding them and dividing by 4, then save the result and allow it to be read back from the file.

```
// frame.getContentPane().add(sturgraderes); // add

sturgraderes = new JTextField(); // grade field
sturgraderes.setBounds(122, 66, 355, 20); // position
frame.getContentPane().add(sturgraderes); // add
sturgraderes.setColumns(10); // width

JLabel Semnum = new JLabel("Semester Number:"); // semester label
Semnum.setFont(new Font("Tahoma", Font.PLAIN, 16)); // font
Semnum.setBounds(10, 118, 142, 14); // position
frame.getContentPane().add(Semnum); // add

semnumres = new JTextField(); // semester field
semnumres.setBounds(148, 117, 330, 20); // position
frame.getContentPane().add(semnumres); // add
semnumres.setColumns(10); // width

JLabel G1 = new JLabel("Grade 1:"); // grade 1 label
G1.setFont(new Font("Tahoma", Font.PLAIN, 16)); // font
G1.setBounds(10, 159, 126, 20); // position
frame.getContentPane().add(G1); // add
```

A lot of JTextFields and labels were used because that is how to build the GUI structure so the user can input their answers.

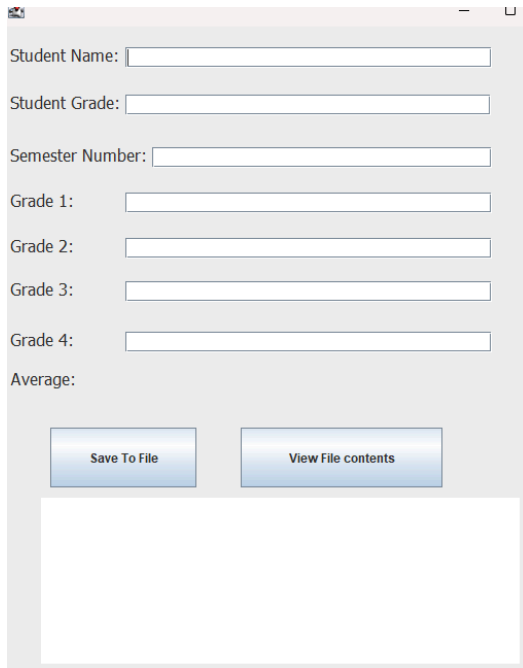
Implementation

Did you write the code in small pieces or attempt the entire solution at once?

I wrote the code in small pieces instead of trying to finish everything at once. First, I focused on building the GUI with the labels, text fields, buttons, and text area. After that, I added the code to calculate the semester average from the four grades. Once that worked, I moved on to saving the student information into the text file, and finally I added the part that reads the file back and displays it in the text area. Working step by step made it much easier to test each section and fix mistakes before adding the next part.

How did you test your solution along the way to make sure it was working?

I tested my solution one part at a time as I built it. First, I checked that the GUI opened correctly and all the text fields and buttons showed up in the right places. Then I tested the average calculation by entering sample grades and making sure the result was correct. After that, I tested the save button to see if the student information was being added to the text file, and finally I used the view file button to confirm that the saved data displayed properly in the text area. Testing each part as I went helped me catch mistakes early and made the full program easier to finish.



The screenshot shows a Java Swing window with a light gray background. It contains several text input fields and two buttons. The fields are labeled: "Student Name:", "Student Grade:", "Semester Number:", "Grade 1:", "Grade 2:", "Grade 3:", "Grade 4:", and "Average:". The "Average:" label is positioned to the left of a large, empty white text area. Below the labels, there are two blue buttons with white text: "Save To File" and "View File contents".

Overcoming Challenges

What part of the problem was the most difficult for you?

The most difficult part of the problem was the file handling section. It was challenging to understand how the program saves the student information into the text file and then reads it back to display in the text area. At first, keeping track of how `FileWriter`, `PrintWriter`, `FileReader`, and `BufferedReader` all worked together was confusing, but after practicing with the Chapter 11 examples and testing it a few times, it became much easier to understand.

```
String line; // current line
String allText = ""; // all file text

while((line = br.readLine()) != null) { //
    allText += line + "\n"; // add line
}
```

I didn't include this part for a while which made me so confused why the program wasn't reading the file until I asked Chatgpt how to fix it

How did you handle moments when you felt stuck or unsure of what to do next?

When I felt stuck, I went back to the Chapter 11 examples and skill builders to compare my code with the file reading and writing examples we practiced before. I focused on one small problem at a time instead of the whole program, like just getting the save button to work before moving to the view button. I also tested the program often after each change, which helped me quickly see what was wrong and figure out the next step.

Learning

Was there anything you learned that you think will help you with future challenges?

Yes, I learned that breaking the problem into smaller parts and testing each part right away makes coding much easier. This will help me in future challenges because instead of getting overwhelmed by the whole program, I can focus on one section at a time, find mistakes faster, and build the solution more confidently.